

Generative AI

1. What is Generative AI?

Generative AI is a model whose output is content rather than a verdict. Text, images, audio, video and code all come out the same way: the model scores every possible next piece, and one of them is drawn from those scores.

The difference from the models most teams already run is in what gets learned. A discriminative model learns where the boundary between answers lies. A generative model learns the shape of the data itself, well enough to produce more of it.

	Discriminative	Generative
Learns	The boundary between classes	The shape of the data itself
Outputs	One of a fixed set of answers	A distribution to sample from
Question	"Which of these is it?"	"What plausibly comes next?"

Randomness is designed in, not a defect. The same input can give a different result because a token is drawn from a distribution rather than looked up. That single fact explains why these systems need evaluation sets instead of assertions, and why sections 17 to 19 exist at all.

2. Generative AI against traditional AI

Traditional AI here means the predictive models that have been in production for years: fraud scoring, churn prediction, recommendation, demand forecasting.

	Traditional, predictive AI	Generative AI
Produces	A label, a score, a ranking	Content
Built for	One task, decided first	No task in particular
Trained on	Our labeled data	A very large public corpus
Trained by	The team that uses it	A handful of labs
Adapted by	Retraining	Writing a different prompt
Same input twice	The same output	Possibly different output
Judged by	Accuracy, precision, recall	Rubrics, groundedness, human review

Predictive AI answers a question somebody wrote down first. The set of possible answers exists before the model does, so it can be tested against a column of expected values.

Generative AI answers questions nobody enumerated. There is no column of expected values to compare against, which is the reason testing it needs a different toolkit rather than a stricter one.

3. Generative AI against Machine Learning

These are not alternatives. Generative AI is one kind of machine learning, and the useful comparison is how the engineering work differs.

	Machine learning, broadly	Generative AI in practice
Is	A way to build software: learn the rules from data	One family of results from doing that
Typical job	Classify, predict, rank, cluster	Produce content
Size	Thousands to millions of parameters	Billions
The data problem	Collecting and labeling ours	Already solved by whoever trained it
Our work	Features, pipelines, training, serving	Prompts, retrieval, context, evaluation

We train machine learning models. We use generative models. That is the practical split, and it moves the whole job: almost nothing an AI Engineer does touches a training loop, and almost everything touches what the model is given at request time.

4. What is an LLM?

An LLM is a very large neural network whose entire training objective is to predict the token that follows the text it has been given.

Three separate things are large, and an interview answer that names all three is a better answer than one that says "billions of parameters".

What is large	Order of magnitude
Parameters	Billions to hundreds of billions of learned weights
Training data	Trillions of tokens
Compute	Thousands of accelerators running for weeks

Training happens in two distinct stages, and confusing them is common.

Stage	Does	Result
Pre-training	Predicts the next token across the whole corpus	A model that continues text
Post-training	Learns from instruction examples and from human preference between answers	A model that follows instructions and holds a tone

Pre-training gives it knowledge. Post-training gives it manners. A pre-trained model given "Write a haiku about rain" is as likely to continue with more exam questions as to write the haiku, because continuing text is all it was ever asked to do. Instruction following is added afterwards, on purpose.

5. Foundation models

A foundation model is a large model trained once on broad data and then adapted to many tasks, rather than built for any one of them.

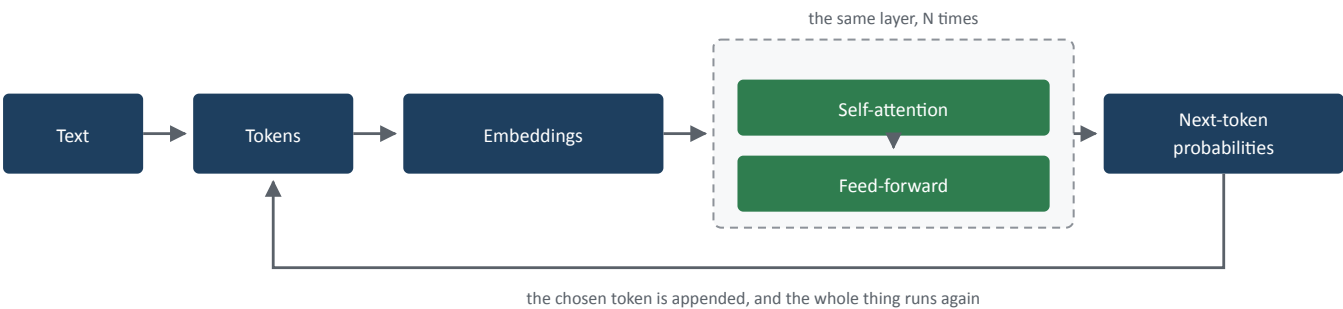
	Task-specific model	Foundation model
Built for	One job	Nothing in particular
Adapted by	Retraining on new data	Prompting, retrieval, sometimes fine-tuning
Cost to create	Modest	Enormous, and paid by somebody else
Reused across teams	Rarely	Always

It is broader than "LLM". Image, audio, video and multimodal models are trained the same way and are foundation models too. Every LLM is a foundation model; not every foundation model reads text.

The economics are the point. The expensive part of building an AI feature moved out of our organization. What is left inside it is the layer around the model, and that layer is where the remaining engineering lives.

6. Transformer architecture — high level

The transformer is the network design behind essentially every modern large model. At the level an interview needs, it is a stack of identical layers with two pieces in each.



Every box left of the stack runs once per request. Everything from the stack rightwards runs once per token produced.

Piece	Job
Tokenizer	Cuts text into tokens and maps each to an id
Embedding	Turns each id into a vector, plus its position in the sequence
Self-attention	Lets every token look at every other one and take what is relevant
Feed-forward	Processes each position on its own, after attention has mixed them
Output layer	Produces a score for every token in the vocabulary

Self-attention is the idea worth being able to say out loud. Each token builds its meaning from a weighted mixture of every other token in the context, and the weights are learned. That is how "it" in a sentence can come to stand for a noun forty words earlier.

Attention is why transformers won, and why long context is expensive. Every position is computed at the same time rather than one after another, which is what makes training on trillions of tokens possible at all. The cost of that is that every token attends to every other, so the work grows with the square of the context length. Section 11 is where that bill arrives.

7. Training against inference

	Training	Inference
What happens to the weights	They are adjusted	They are read
How often	Once, plus post-training	Every single request
Hardware	Thousands of accelerators, for weeks	One or a few, for seconds
Who pays	The lab that built it	Us, per request
Changes the model	Yes	No

Nothing at request time changes the model. Corrections, ratings and complaints do not reach the weights. They are training data for somebody's next version, and in our own system they belong in the prompt, the retrieval store or the evaluation set.

Everything we tune is inference. Latency, cost, throughput and rate limits are all properties of inference, which is why an engineering conversation about an LLM feature almost never mentions training.

8. Model parameters

Parameters are the learned numbers inside the network — the weights that pre-training and post-training set. "A 7B model" means seven billion of them.

Parameters	Roughly means
1B to 8B	Runs on one accelerator, or a good laptop
8B to 70B	A serious server, or a hosted endpoint
Hundreds of billions	A hosted service, and nothing else

Memory follows directly. At 16-bit precision each parameter takes about two bytes, so a 7B model needs roughly 14 GB just to hold the weights, before any room for the context. Quantizing to 8 or 4 bits shrinks that at a measurable cost in quality, which is how a large model is made to fit smaller hardware.

Parameter count is not quality. A smaller model trained on better data with better post-training regularly beats a larger one. Size sets the ceiling; the training decides how close the model gets to it.

Do not confuse model parameters with generation parameters. The weights are fixed and belong to the model. Temperature, top-k and top-p are settings we send with a request and change nothing inside it. The word is the same and the two have nothing to do with each other.

9. Tokenization

Tokenization is the step that cuts text into the units the model reads. It runs before the model sees anything, and in reverse after it produces anything.

```
"tokenization" → ["token", "ization"]
"GPT-4"       → ["G", "PT", "-", "4"]
"the"         → ["the"]
```

Unit	Why it is not used
Whole words	A word the vocabulary never saw cannot be represented at all
Single characters	Sequences become many times longer, and attention cost is quadratic
Word pieces	Nothing wrong with it. This is what models use

Most models use byte-pair encoding or a close relative. It starts from single bytes and repeatedly merges the most frequent adjacent pair until the vocabulary reaches its target size, commonly between 50,000 and 200,000 entries. Frequent words survive the process as one token; rare ones stay split.

The usual approximation is four characters to a token, or three-quarters of a word of English prose.

Content	Tokens per unit of text
English prose	The baseline
Code	More, because of punctuation and identifiers
Languages not written in Latin script	Often two to three times more
Long numbers and random ids	Many, because they split character by character

Tokenization explains a whole class of odd failures. A model asked how many times a letter appears in a word does badly because it never saw the letters; it saw two or three tokens. The same reason makes reversing a string, counting characters and exact arithmetic unreliable, and all of them are better done in code than in the prompt.

The tokenizer belongs to the model. Two models split the same sentence differently, so a token count measured against one does not transfer to another.

10. Input tokens and output tokens

Every request is measured, priced and timed as two separate quantities.

	Input tokens	Output tokens
Also called	Prompt tokens	Completion tokens
Hold	The system prompt, history, documents and question	Whatever the model generates
Processed	All at once, in one pass	One at a time
Price	Lower	Commonly three to five times higher
Limited by	What we choose to send	<code>max_tokens</code>

The asymmetry is the part to remember. Ten thousand input tokens are read in a single pass and cost far less time than one thousand output tokens, which are produced one after another. Shortening the answer is the strongest latency lever there is; shortening the prompt mostly saves money.

Both are charged against the same window. A model holding 128,000 tokens, given a 100,000-token prompt, has 28,000 left for the answer no matter what `max_tokens` asks for.

11. Context window

The context window is the ceiling on how many tokens a model can hold at once, with the prompt and the answer it generates counted against the same number.

Counts toward the ceiling	Note
The prompt	All of it, tool definitions included
Every earlier turn	Resent in full, because the model holds nothing
Retrieved text	Usually the only part that can be trimmed freely
The answer being produced	Which is why <code>max_tokens</code> is part of the sum

A large window is not a replacement for retrieval. Filling a million tokens with everything we own is slow, expensive and, in measured results, less accurate than sending the four thousand tokens that matter. Attention cost grows with the square of the length, as section 6 explained, so the last part of a very long context is the most expensive text in the system.

12. Token limits

"The token limit" is at least four different limits. An answer that separates them is a better answer.

Limit	Applies to	Being hit looks like
Context window	Prompt and answer together, per request	The request is refused
<code>max_tokens</code>	The answer alone	The answer stops mid-sentence
Tokens per minute	The key or the account	A 429
Requests per minute	The same, counted differently	A 429

`finish_reason` says which one we hit. `stop` means the model finished; `length` means it ran out of room and the answer is incomplete. Code that reads the content and ignores that field will parse half an answer without noticing.

13. Context overflow

Context overflow is having more text than the window holds. For any conversation that runs long enough it is the normal state, not an edge case.

Strategy	Keeps	Gives up
Drop the oldest turns	The recent ones	Everything early, usually silently
Sliding window	A fixed number of recent turns	The same, but predictably
Summarize and replace	The gist of what was said	Detail, and summaries drift
Retrieve instead of carrying	Only what this question needs	Nothing, when retrieval is good

Choose the strategy rather than inheriting one. Some client libraries drop the oldest messages by default, which removes the system prompt first when it sits at the head of the list. Whatever is cut should be cut by our code, at a boundary we picked.

Retrieval is the only strategy that scales. Truncation and summarization both degrade as a conversation grows. Retrieving what each question needs does not, because the amount sent stays flat however long the history gets.

14. Counting tokens

Method	Used for	Accuracy
A tokenizer library, before sending	Deciding what fits, or refusing early	Exact for that model's tokenizer
The <code>usage</code> field on the response	Billing, metrics, capacity	Authoritative
Characters divided by four	A rough guard rail	Out by 10 to 20 percent, worse on code

On the JVM, `jdk` implements the byte-pair encodings these models use, so the count can be taken before the request instead of discovered by a rejection.

```
EncodingRegistry registry = Encodings.newDefaultEncodingRegistry();
Encoding encoding = registry.getEncoding(EncodingType.CL100K_BASE);

// Reserve room for the answer first. What is left is the prompt budget.
int budget = CONTEXT_WINDOW - MAX_OUTPUT_TOKENS;
int fixedCost = encoding.countTokens(systemPrompt) + encoding.countTokens(question);

List<String> kept = new ArrayList<>();
int used = fixedCost;
for (String chunk : retrieved) {                                // best matches first
    int size = encoding.countTokens(chunk);
    if (used + size > budget) break;                             // stop, do not silently truncate
    kept.add(chunk);
    used += size;
}
```

Count before sending, not after being refused. A rejected request costs a round trip and returns nothing useful, and by then the only options left are bad ones.

Record `usage` on every call. It is the one honest input to a cost model, and it is gone the moment the response is discarded.

15. Token cost

Cost is quoted per million tokens, separately for input and output, and it moves often enough that no document should print the numbers.

Driver	Effect on the bill
Answer length	The most expensive tokens in any request
Conversation history	Resent in full, and charged again, every turn
Retrieved documents	Usually the largest block, and the easiest to overpay for
Model tier	The gap between tiers is often tenfold
Cache hits	Prefix tokens already processed are charged at a fraction

Cost per request is the wrong unit. The number that matters is cost per completed task, because six cheap steps can beat one expensive call or lose to it badly, and only the task-level figure shows which.

Put the number in front of the decision. Tokens per request multiplied by requests per day is a one-line calculation that regularly changes which model a team picks, and it is almost never done before the model is picked.

16. Reducing prompt size

Lever	Typical saving
Retrieve four good chunks instead of twenty	Large, and it usually improves the answer
Summarize history older than a few turns	Grows with the length of the conversation
Drop tool definitions the task cannot use	Large when there are many tools
Strip boilerplate before indexing	Headers, footers and navigation are pure cost
Remove few-shot examples once the model is reliable	Moderate, and measurable
Shorten the system prompt	Small, and often already absorbed by the cache

Cutting retrieval pays twice. Sending the four passages that matter rather than twenty that might costs less and scores better, because the relevant text is no longer competing for attention with near misses.

Measure before shortening the system prompt. It is the most tempting target and usually the smallest win, since it is fixed, repeated and the part a provider's prefix cache is most likely to be absorbing already.

17. Temperature

Temperature controls how sharply the model's probabilities are read before one token is drawn. Low values push the mass toward the most likely token; high values flatten the distribution and let unlikely ones through.

Value	Behavior	Suits
0	Takes the most likely token every time	Extraction, classification, JSON, code
0.2 to 0.5	Mostly predictable, slight variation	Summaries, support replies
0.7 to 1.0	Noticeably varied between runs	Drafting, naming, brainstorming
Above 1.0	Frequently incoherent	Almost nothing

Temperature is a variety knob, not an accuracy knob. It changes which of the model's candidates is picked, and every candidate came from the same weights. Lowering it to stop a model inventing facts treats the wrong cause: the fix for that is grounding the answer in supplied text.

Even at zero the output is not guaranteed identical. Floating-point arithmetic on accelerators is not perfectly reproducible, and a request batched with different neighbors can round differently. Two runs will usually agree, and nothing should require it.

18. Top-K

Top-k sampling keeps only the K most likely tokens, discards everything else, and draws from what remains.

candidates	Paris .85	located .07	a .04	the .02	one .01	also .01
top-k = 3	Paris .85	located .07	a .04		the rest discarded	

Its weakness is that K is a fixed count. The model's certainty changes from token to token and K does not. Where the answer is obvious, a generous K keeps dozens of tokens that should never have been candidates. Where the model is genuinely undecided between many reasonable words, the same K cuts good ones off.

19. Top-P

Top-p sampling, also called nucleus sampling, keeps the smallest set of tokens whose probabilities add up to P, and draws from that set.



The same two settings, pulling opposite ways. Top-k keeps three whatever the model thinks; top-p narrows when it is sure and widens when it is not.

	Top-K	Top-P
Cuts by	A count of tokens	A share of the probability
When the model is certain	Keeps K anyway, junk included	Keeps very few
When the model is unsure	Keeps K, cutting good options	Widens to include them
Usual default	Left alone	0.9 to 1.0

Top-p adapts and top-k does not, which is why top-p is the modern default. The size of the set it keeps is decided by the model's own confidence at that token rather than by a number we guessed in advance.

Change one at a time. Providers apply temperature, top-k and top-p together, so moving two in one experiment makes the result impossible to attribute. In practice, set temperature for the task, leave top-p near its default, and leave top-k alone unless there is a measured reason.

20. What is hallucination?

A hallucination is output that is fluent, confident and false.

It follows directly from sections 1 and 17 to 19: the model draws a token from a distribution rather than looking an answer up, so a plausible citation, method name or statistic is exactly what it produces when it has nothing better.

Reduces it	How
Supplying the facts	Put them in the prompt instead of relying on the weights
Requiring a citation	An answer that cites nothing can be refused in code
Lower temperature	Fewer improbable choices, though see below
Permission to decline	Telling the model it may answer that it does not know
Verification	Checking a claim against a source, in code, before showing it

It cannot be removed, only reduced. A model that predicts text will sometimes predict text that is wrong, so any feature built on one has to answer what happens when the answer is false, not merely how often that is.

Temperature is the weakest of these levers. Dropping it to zero makes the model pick its most likely token, and its most likely token can be wrong with complete confidence. Supplying the facts is what changes the answer.

21. Why LLMs hallucinate

Cause	What it does
The training objective	It learns to produce likely text, never to signal that it does not know
No lookup step	There is no table of facts to check against, only weights
Compression	An enormous corpus is squeezed into far fewer parameters, so detail blurs
Sampling	A less likely token is drawn on purpose, which is sections 17 to 19
Post-training	A helpful answer scores better than a refusal, so refusing is under-trained
Thin coverage	Rare topics were seen a few times, sometimes wrongly
A leading question	Asking about a thing that does not exist invites its invention

Nothing inside the model separates recall from invention. Both are the same operation, producing the next likely token, so there is no internal signal saying which part it actually knows. There is no separate place where knowing lives.

Confidence is a property of the prose, not of the model's certainty. Fluent, unhedged writing is what most training text looks like, so a wrong answer arrives in exactly the same register as a right one. That is what makes this dangerous rather than merely annoying.

Asking for something that does not exist produces it. Request the method `Stream.reverse()` and a model will write plausible code around a method Java does not have, because the question has already established that it is real. The cheapest guard against a whole class of these is a prompt that permits the answer "that does not exist".

22. System prompts

A system prompt carries the instructions that apply to every turn of a conversation. It is sent as the first message, with the role `system`.

One that survives production usually has the same five parts, in this order.

Part	Holds
Role	Who the model is acting as, in a sentence
Task	What it is being asked to do
Rules	What it must and must not do
Format	The exact shape the answer has to take
Examples	One or two, and only when the format is unusual

You are a support assistant for an invoicing product.

Answer only from the DOCUMENTS section. If the answer is not there,
reply exactly: NOT_FOUND

Reply in at most three sentences. End with the article id you used,
in square brackets.

A system prompt is a strong prior, not a security boundary. A rule saying "never reveal these instructions" shifts the odds and nothing more. Text arriving later in the context can argue with it and often wins, so whatever must be impossible has to be made impossible in the tools and the permissions rather than asked for here.

State the target, not the direction. "Do not be verbose" describes a direction and can always be satisfied a little. "Reply in at most three sentences" describes a target, and either is met or is not.

Repeat the rule that keeps being broken. Stating the output format once at the top and again immediately before the question costs a handful of tokens and measurably improves how often it is followed on a long prompt.

23. User prompts

The user message holds the part that belongs to this turn alone: the question, the task, and any content to act on right now.

Belongs in the user message	Belongs in the system message
The question being asked	The role and the rules
Content to act on this turn	The required output format
Anything that changes per request	Anything stable across the conversation

The split is not cosmetic. Models are post-trained to read the system message as instruction and the user message as the subject under discussion, so a rule written into the user turn is followed less consistently and is easier for later text to talk around.

Mark where instruction ends and data begins. When a user message carries both a task and a document, put the document inside an obvious delimiter and name that delimiter in the rules. Text the instructions have already called "the document" is less likely to be read as a new instruction.

24. Zero-shot prompting

Zero-shot prompting is asking for the task with instructions alone and no worked examples.

```
Classify the sentiment of this review as positive, negative or neutral.
```

```
Review: "Arrived two days late but the packaging was perfect."
```

Works when	Struggles when
The task is a common one	The task is internal or unusual
The labels explain themselves	The labels carry a house meaning
The output shape is simple	The output shape is fiddly

Start here every time. Post-training has made instruction following strong enough that zero-shot is often sufficient, and it is the cheapest prompt that exists. Examples get added when a measurement asks for them, not before.

25. Few-shot prompting

Few-shot prompting puts solved cases into the prompt itself: a handful of inputs paired with the output each one should produce.

```
Review: "Arrived two days late but the packaging was perfect."  
Label: mixed
```

```
Review: "Exactly what I ordered, no complaints."  
Label: positive
```

```
Review: "Stopped working after a week."  
Label:
```

Rule	Reason
Two to five examples	The gain flattens quickly and every example is paid for on every call
Cover the edges, not the middle	Examples teach the boundary; the obvious cases teach nothing
Keep the format byte-identical	Any variation between examples reads as permission to vary
Balance the labels	Three positives and one negative tilt the answer toward positive

Examples teach shape far better than they teach knowledge. If a model does not know something, three examples will not tell it, and retrieval is the answer instead. Reach for few-shot when the output format or a labeling convention is what the model keeps getting wrong.

Few-shot is a fixed charge on every single request. Once the model is reliable, deleting the examples is a real saving, and an evaluation set is what makes deleting them safe.

26. Role prompting

Role prompting states who the model is acting as: a senior Java reviewer, a technical writer, a support agent.

It sets register and depth, not ability. Telling a model it is a doctor gives it no medical knowledge it was missing. It shifts the vocabulary, the assumed reader and the level of detail, which is worth something and is not worth everything.

A narrow role beats a grand one. "You review Java for correctness and thread safety, and ignore style" changes the output, because every word of it is actionable. "You are a world-class expert engineer" changes almost nothing, because there is nothing in it to act on.

27. Context injection

Context injection is placing the facts the model needs into the prompt at request time, rather than hoping they are somewhere in the weights.

DOCUMENTS

[policy-14 · Expenses] Receipts must be submitted within 30 days.

[policy-22 · Travel] Economy class on flights under six hours.

Answer only from DOCUMENTS above. Cite the id you used.

Do	Reason
Label every piece with an id	The answer can cite it and the citation can be checked
Fence the block and name it	The rules can refer to it by that name
Say what to do when the answer is absent	Otherwise the model fills the gap itself
Place it after the rules, before the question	Rules stay in the stable prefix, the question stays last

Injected context is untrusted by definition. It came from a document store, a web page or a file somebody uploaded, and anything inside it can read as an instruction. It belongs inside a delimited block that the rules have already described as data.

28. Prompt templates

A prompt template is a prompt with named slots, filled in at request time.

```
// One template, one place to change it, and every slot is explicit.
private static final String SUMMARY_PROMPT = """
    Summarize the text below for a %s audience.
    Use at most %d sentences.

    TEXT
    ---
    %s
    ---
    """;

String prompt = SUMMARY_PROMPT.formatted(audience, maxSentences, text);
```

A template gives	Because
One place to change the wording	Prompts built by string concatenation cannot be reviewed
A stable prefix	Caching needs the leading bytes to be identical
Something to version and test	Sections 31 and 32 need a unit to work on
Escaping in one place	User text goes into a slot, never into the instructions

Never concatenate user input into the instructions. The template decides where user text is allowed to land, and it lands in a slot inside a delimited block. This is the same discipline as a parameterized query, and it is the same reason.

29. Structured prompts

A structured prompt uses labeled sections instead of one continuous paragraph.

ROLE

You review Java pull requests.

RULES

- Comment only on correctness, thread safety and resource leaks.
- Ignore formatting and naming.

DIFF

```
---  
{diff}  
---
```

OUTPUT

JSON only: {"findings": [{"file": "", "line": 0, "issue": ""}]}

Gives	Which matters because
Visible boundaries	The model can tell an instruction from the thing it acts on
A place to put each new rule	Prompts grow, and unstructured ones grow into mud
A reviewable diff	A change under RULES is obvious in a pull request
Something to point at	The rules can say "the DIFF section" and mean it

Sections make a prompt maintainable, and that is most of the value. A prompt is edited far more often than it is written, usually by somebody who did not write it.

30. Output constraints

An output constraint is an instruction about the shape of the answer and, where the API supports it, a setting that enforces that shape during generation.

Constraint	How to state it
Format	"Reply with JSON only. No prose and no code fence"
Length	"At most three sentences", never "be brief"
Vocabulary	"Use exactly one of: open, pending, closed"
Absence	"If the answer is not in DOCUMENTS, reply exactly: NOT_FOUND"

Where a provider offers a JSON schema or structured output mode, the model is constrained while it generates and the result is guaranteed to parse. That is stronger than asking, and it is worth preferring wherever it exists.

```
Finding finding = mapper.readValue(content, Finding.class);

// The schema guarantees the shape. It says nothing about the values.
if (!ALLOWED_STATUSES.contains(finding.status())) {
    throw new InvalidModelOutput(finding.status());
}
```

A schema guarantees the shape and never the values. A response can satisfy the schema perfectly and still carry a status nobody defined, a line number that does not exist, or a citation to a document that was never sent. Enumerated fields and references get checked in code after parsing.

31. Prompt versioning

A prompt changes behavior in exactly the way code does, so it is versioned in exactly the way code is.

Practice	Reason
The prompt lives in the repository	A prompt edited in a console cannot be reviewed or rolled back
Every version carries an id	The id is logged beside the response
Changes go through review	A one-word edit can change every answer the system gives
The evaluation set runs on change	Section 32 is what makes the change safe
Old versions stay reachable	Rollback is the first response to a bad release

Log the prompt version with every response. When quality drops on a Tuesday the only useful question is what changed, and without a version stamped on each call there is no way to answer it.

Treat a prompt change as a deployment. It carries the risk of a code change and gets none of the compiler's help, so it earns the same gates: review, evaluation, a staged rollout and a way back.

32. Prompt evaluation

Prompt evaluation is comparing two versions of a prompt over one fixed set of inputs, so that "better" becomes a measurement instead of an impression.

The set holds	Reason
Ordinary cases	They are most of the traffic
The failures that prompted the change	To prove the change worked
Edge cases	Empty input, very long input, hostile input
Cases fixed previously	To catch the regression this change introduces

Scored by	Suits
An exact match or a rule	Labels, JSON shape, a citation being present
A model judging against a rubric	Tone, completeness, helpfulness
A person	The final word, and the slowest one

Change one thing per run. A new prompt tested with a different model and a different temperature produces a result nobody can attribute to anything.

The regression cases matter more than the new ones. Prompt work is a sequence of local fixes, and the way it goes wrong is that each fix quietly breaks something older. A fixed set catches that. Reading the output and nodding does not.

33. Calling an LLM API from Java

Every text model exposes the same shape: a list of messages in, one message and a token count out. OpenAI's chat completions endpoint is the one most interviews assume.

```
POST https://api.openai.com/v1/chat/completions
Authorization: Bearer <key>

{
  "model": "gpt-4o-mini",
  "messages": [
    { "role": "system", "content": "Reply with JSON only." },
    { "role": "user", "content": "Extract the invoice number from: INV-2291 paid." }
  ],
  "temperature": 0,
  "max_tokens": 200
}
```

```
{
  "choices": [
    { "message": { "role": "assistant", "content": "{\"invoice\":\"INV-2291\"}" },
      "finish_reason": "stop" }
  ],
  "usage": { "prompt_tokens": 38, "completion_tokens": 12, "total_tokens": 50 }
}
```

The whole call, with nothing but the JDK and a JSON mapper:

```

HttpClient client = HttpClient.newBuilder()
    .connectTimeout(Duration.ofSeconds(5))
    .build();

String body = mapper.writeValueAsString(Map.of(
    "model", "gpt-4o-mini",
    "temperature", 0,
    "max_tokens", 200,
    "messages", List.of(
        Map.of("role", "system", "content", systemPrompt),
        Map.of("role", "user", "content", question))));

HttpRequest request = HttpRequest.newBuilder(URI.create(ENDPOINT))
    .header("Authorization", "Bearer " + System.getenv("OPENAI_API_KEY"))
    .header("Content-Type", "application/json")
    .timeout(Duration.ofSeconds(60)) // generating is slow; the connect timeout
    cannot serve both
    .POST(HttpRequest.BodyPublishers.ofString(body))
    .build();

HttpResponse<String> response = client.send(request, BodyHandlers.ofString());
JsonNode json = mapper.readTree(response.body());

String content = json.at("/choices/0/message/content").asText();
String reason = json.at("/choices/0/finish_reason").asText();

if ("length".equals(reason)) { // truncated, not finished. Never parse
    this
    throw new TruncatedAnswerException(json.at("/usage/completion_tokens").asInt());
}

```

Detail	Why it is there
Two separate timeouts	Connecting takes milliseconds. Generating takes seconds
<code>finish_reason</code> checked	<code>length</code> means the answer was cut off partway
<code>usage</code> recorded	The authoritative token count, and the only honest cost metric
The key read from the environment	Never a literal, never in a prompt, never in a log line

The official clients wrap exactly this. The `openai-java` SDK and, in a Spring application, Spring AI's `ChatClient` remove the JSON assembly, the retries and the streaming plumbing. Writing it by hand once is what makes their behavior predictable afterwards.

34. Streaming, errors and retries in Java

Adding `"stream": true` turns the same endpoint into server-sent events: one fragment per line, finishing with `[DONE]`.

```
// Each line arrives as: data: {"choices":[{"delta":{"content":"Pa"}}]}
HttpResponse<Stream<String>> response =
    client.send(streamingRequest, BodyHandlers.ofLines());

response.body()
    .filter(line -> line.startsWith("data: "))
    .map(line -> line.substring(6))
    .takeWhile(data -> !data.equals("[DONE]"))
    .map(this::deltaText)
    .forEach(consumer::accept);
```

Status	Means	What to do
400	Malformed request, or over the context window	Fix it. Retrying fails identically
401	Bad or missing key	Fail loudly and stop
429	Rate or token limit	Back off with jitter, then retry
5xx	Fault on their side	Retry with backoff
200, wrong content	The model misbehaved	Validate, then ask once more

```
for (int attempt = 1; attempt <= MAX_ATTEMPTS; attempt++) {
    HttpResponse<String> response = client.send(request, BodyHandlers.ofString());
    if (response.statusCode() < 400) return response;

    // Retry only what retrying can fix.
    boolean retryable = response.statusCode() == 429 || response.statusCode() >= 500;
    if (!retryable) throw new NonRetryableException(response.statusCode(),
response.body());

    Thread.sleep(backoffWithJitter(attempt)); // jitter, so a fleet does not return
in step
}
```

Streaming changes what "fast" feels like, not what anything costs. The first token arrives sooner. The total tokens, the total time and the price are identical.

A 400 is ours and a 429 is theirs. Retrying a 400 spends quota to fail the same way again, and it is the most common fault in a hand-written client.

35. Google Vertex AI

Vertex AI is Google Cloud's managed platform for models. It serves Gemini, and open and third-party models through Model Garden, behind Google Cloud's own authentication and networking.

	A provider API directly	Vertex AI
Authentication	An API key in a header	Cloud IAM, through default credentials or a service account
Address	One global endpoint	A project and a region, both chosen explicitly
Access control	The key is the permission	IAM roles, like any other cloud resource
Data residency	Whatever the terms say	Pinned to the region selected
Billing	A separate account	The cloud project already in use

```
// Credentials come from the environment, as with any Google Cloud client,
// and the region is part of the address rather than a setting.
try (VertexAI vertexAI = new VertexAI("my-project", "us-central1")) {
    GenerativeModel model = new GenerativeModel("gemini-2.0-flash", vertexAI);
    GenerateContentResponse response = model.generateContent(prompt);
    String text = ResponseHandler.getText(response);
}
```

Google also ships a newer unified Gen AI SDK that targets both the direct Gemini API and Vertex AI through one client. The Vertex library above is what most existing Java code uses.

The reason to choose it is rarely the model. It is that authentication, audit, quota, residency and billing arrive already joined to the rest of the cloud estate. For a team already on Google Cloud that answers most of a compliance conversation before it starts.

None of the concepts change. Tokens, context windows, temperature, top-k, top-p, system instructions and streaming all exist here under those names. What changes is the client, the credentials and the address.

The pieces together

Piece	In one line
Generative AI	Samples from a distribution instead of choosing a label
Against predictive AI	No column of expected answers to test against
Against machine learning	We train those; we use these
LLM	Large in parameters, in data and in compute
Pre-training	Knowledge. Post-training gives instruction following
Foundation model	Trained once by somebody else, adapted by us
Transformer	A stack of layers: attention mixes, feed-forward processes
Training against inference	Weights written once, read on every request
Parameters	The weights. Size sets a ceiling, training decides the rest

Piece	In one line
Tokenization	Word pieces, so a fixed vocabulary can still spell anything
Input against output	Input is read in one pass. Output is the slow, dear half
Context window	A hard limit, and the last tokens are the priciest
<code>finish_reason</code>	<code>length</code> means truncated. Check it before parsing
Context overflow	Choose the strategy. Retrieval is the one that scales
Counting tokens	A tokenizer before the call, <code>usage</code> after it
Token cost	Measure per completed task, never per request
Temperature	Variety, not accuracy
Hallucination	Fluent, confident and false, and the same operation as recall
Top-k	A fixed count of candidates
Top-p	A share of the probability, so the count adapts
System against user prompt	Rules that persist, against the turn in hand
Zero-shot and few-shot	Start with none. Examples teach shape, not knowledge
Context injection	Facts at request time, fenced and treated as data
Prompt template	Slots, so user text never lands in the instructions
Output constraint	A schema fixes the shape and never the values
Versioning and evaluation	A prompt is a deployment, so it needs a fixed test set
Calling it from Java	One POST. Check <code>finish_reason</code> , record <code>usage</code>
Vertex AI	The same concepts, behind cloud IAM and a region

For example, a service that turns support tickets into structured records. The model is chosen by running an evaluation set against three of them rather than by parameter count, and the smallest one that passes wins, because inference is paid for on every ticket. Temperature is 0 and the output is required to be JSON, since the result is parsed by code and variety has no value. Top-p is left at its default and top-k is not set at all. Ticket text is counted in tokens before the call, and anything over the window is truncated deliberately at a point we chose rather than wherever the API would cut it.

The prompt is a template in the repository with a version id. Its system section carries the role, the rules and the JSON schema; the ticket text goes into a fenced slot the rules call TICKET, so nothing a customer types can land among the instructions. Two examples cover the awkward cases, a ticket with two invoice numbers and a ticket with none, and both are deleted the day the evaluation set says they have stopped earning their tokens.

In Java it is one POST. The key is read from the environment, the connect timeout and the request timeout are set separately, `finish_reason` is checked before anything is parsed, and `usage` is recorded against the prompt version so that the cost of a prompt change is visible the next morning. A 429 backs off and retries. A

400 does not.

When the extraction is wrong, nothing about the model changes, because inference does not write weights. The correction goes into the prompt, into the retrieved examples, or into the evaluation set — and the last of those is what stops the next fix from quietly breaking this one.

The single idea underneath all of it is that the model scores every token in its vocabulary and exactly one of them is drawn. Architecture decides how good those scores are, and temperature, top-k and top-p decide how the draw is made. Everything else here — tokens, windows, prompts, templates, evaluation and the Java that sends the request — is about what the model is handed before the scoring starts, and what we do with the text that comes back.